

WEEK 13

GraphRAG

跨文档关系与全局归纳——让 RAG 从“答得对”升级到“能跨文档归纳 + 多跳推理”，但只在主场题型发力

01 Boundary

边界识别

何时上图 · 5 判断标准

02 Build

图构建

schema-first + 实体对齐

03 Search

图检索

Local / Global / Multi-hop

04 Augment

图增强生成

图→文本 + 3 套模板

05 Compare

A/B 对比

按题型分层 + 治理纳入

本周划重点

Week 8 我们把向量 RAG 做到了“答得对”。可有一类问题它一定翻车——“过去半年所有 P0 故障的共性是什么”。答案不在某几个 chunk 里，在几百个 chunk 的关系里。这周我们给 RAG 补上“看关系、做归纳”的能力，但我先把话撂这儿：GraphRAG 不是万能药，它只在主场题型上才回得来本。



LESSON 01 · WHEN TO USE GRAPH RAG

GraphRAG 的边界：什么时候需要图，什么时候纯向量就够

Week 8 做完了"混合检索 + 重排 + 受约束生成"——这一节先解决一个比技术更重要的问题：边界

真实场景 · 一句问话把向量 RAG 打回原形

我带团队做过一个客服知识库，向量 RAG 上线三个月，FAQ、故障排查答得都挺漂亮，客户满意度也上来了。直到有天业务总监丢来一句："过去半年，所有 P0 故障的共性是什么？" 系统吭哧半天，返回了 3 条"最相似"的工单——牛头不对马嘴。那不是模型笨，是范式不匹配：这个问题的答案，压根不在某几个 chunk 里，它散在 100 多份文档的"关系"里。这一节我把这条边界讲透——什么时候该升级到 GraphRAG，什么时候上图纯属给自己找麻烦。

咱们这节聊 4 件事：

- 为什么"答得不准就上 GraphRAG"是最常见的一笔工程债
- 向量 RAG 在哪 4 类题型上几乎一定失败、又在哪 3 类场景上千万别上图
- GraphRAG 的真实成本账——以及 2026 年 LazyGraphRAG 怎么把这笔账重写了
- 上图前的 5 个判断标准：全过才上、只过 3 项先做 PoC



NOT A BETTER RAG

GraphRAG 不是"更好的 RAG"——它是为一类向量 RAG 必然失败的题型而生

搞错这个定位，你要么该上不上、要么不该上瞎上

向量检索有一个藏在水面下的工程假设：答案藏在"语义相近的某几个 chunk"里。这个假设在 70% 的客服 / 知识问答场景成立——这也是为什么 RAG 这套范式这几年能跑通、能落地。

向量的主场

- "怎么重置密码"
- "什么是 SLA"
- 答案就在 1-2 个 chunk
- 语义相似 = 找得到
- 这类占大多数

向量的死角

- "半年 P0 故障的共性"
- "列出所有提到 X 的决策"
- "A 收购 B，B 的母公司在做啥"
- 答案在 chunk 之间的关系
- 语义相似 = 找不到

GraphRAG 的起点

- 把"关系"显式建成图
- 检索能跨越 chunk 边界
- 生成能在全局结构上归纳
- 这是它唯一存在的理由
- 不是替代，是补位

WHEN VECTOR FAILS

向量 RAG 几乎一定失败的 4 类题型——记住这张表就够了

它们有个共同点：答案的"完整性"依赖关系 / 时间 / 多源，而不是局部语义相似

题型	具体问法	为什么向量必失败	GraphRAG 怎么解
全局归纳	"过去 6 个月主要故障类型"	答案不在某几个 chunk，散在 100+ 文档	社区检测 + 主题聚合
关系遍历	"A 公司的子公司的供应商有哪些"	多跳关系向量编码不了	图路径遍历
交叉验证	"X 在合同/邮件/工单里各怎么描述"	同一实体跨文档语义不一致	实体对齐 + 多源聚合
时间序列	"产品 V1→V2→V3 的演进"	时序关系向量会丢	时间属性的关系边

关键判断

这 4 类题型你一眼就能认出来：问题里带"所有 / 共性 / 分布 / 演进 / 之间的关系"这类词，基本就是向量的死角。我的经验是——先把业务的历史问题日志拉出来，数一数这 4 类占多少，这个比例决定你到底该不该上图。



WHEN VECTOR WINS

向量 RAG 的主场——这些场景千万别上 GraphRAG

上图 = 给自己加 3-10 倍成本，还引入一堆噪声，纯属过度工程

FAQ / 操作类

- "怎么重置密码"
- "忘了用户名怎么办"
- 答案就在 1-2 个 chunk
- 向量召回足够
- 上图 = 过度工程

定义 / 概念类

- "什么是 SLA"
- "年化收益率怎么算"
- 单点知识查询
- 向量天然擅长
- 加图反而引入噪声

短对话类

- 客服对话 / 闲聊
- 上下文短、关系简单
- 不需要复杂图建模
- 向量 + Tool 足够
- 图在这儿一点用没有

踩坑提醒

我见过太多团队，看了篇 GraphRAG 的论文就兴冲冲全量上图，结果 80% 的查询是 FAQ 类——图一点没帮上忙，还把每次查询延迟从 800 毫秒拖到 3 秒、成本翻了 5 倍。记住：不该上图的场景硬上，比不上图更糟。

THE 2024 COST ACCOUNT

GraphRAG 的成本账（2024 版）——每一项都比向量贵 3-10 倍

这是过去两年所有人吓退的原因，先看清楚，下一页我给你 2026 的新账

成本类别	主要开销	相对向量	2024 典型量级
构建（一次性）	LLM 逐 chunk 抽实体 + 关系	5-10x	每 1k chunk 约 \$50-200
存储（持续）	图 DB + 索引 + 元数据	2-3x	同等数据存储翻倍
更新（增量）	增量抽取 + 图结构变更	5-10x	每次 update 触发图重算
查询（每次）	图遍历 + LLM 二次推理	3-5x	每查询 +1-3s 延迟

关键判断

2024 年，微软 GraphRAG 官方 demo 里，给一个数据集建一次全量索引要 \$33,000——不是打错字，三万三千美金。这就是为什么当年"要不要上 GraphRAG"是个让 CTO 肉疼的决定。但这笔账，2026 年已经被彻底重写了。

THE 2026 COST CLIFF

2026 的成本悬崖：LazyGraphRAG 把索引成本打到"和向量一样"

微软 2024.11 放出的 LazyGraphRAG——这是重新决定"要不要上图"的关键变量

方案	索引成本	全局查询成本	质量
Full GraphRAG (2024)	基准 100% (\$33k 那套)	高 (每次全图 Map-Reduce)	高
LazyGraphRAG (2024.11)	≈ 向量 (每千 chunk < \$1) · full 的 0.1%	比 Global Search 低 700 倍	局部题追平、全局题相当
纯向量 RAG	最低	最低	归纳/多跳题几乎为 0

老司机解读

LazyGraphRAG 的思路很"老司机"——别在建图时就把 LLM 的钱全烧掉，把重活推迟到查询时按需做。它建图只用便宜的 NLP 抽名词短语 + 社区结构，LLM 留到用户真问了才上。结果就是那句业内名言："\$33,000 → \$33, 18 个月。"

给你的落地判断：2026 年再拿 2024 那套"贵 5-10 倍"的成本账吓自己，已经过时了。先上 LazyGraphRAG 试水，它索引跟向量一样便宜，你几乎没有理由不试——把"要不要上图"从一道成本难题，变成了一道"值不值得多花那点查询钱"的小题。

GRAPH AS DERIVED ASSET

GraphRAG 是"派生数据资产"——建在 Week 6-7 数据工厂之上，不是另起炉灶

这是我见过的第二大坑：另搭一套抽取流水线、另选一个图库、另起一个服务

很多团队第一次做 GraphRAG，条件反射就是"重起炉灶"——另搭抽取、另选图库、另起查询服务，结果图跟原有数据资产两张皮，契约、版本、血缘、评测全断了。正确做法：把图建成 Week 6 数据工厂的"下游派生产物"——Source: silver.chunks → derived: entity_node + relation_edge。

契约继承

- 图 schema 也进契约
- 强类型 + 版本化
- Week 2 契约能力直接用

血缘继承

- 每个实体带 source_chunk_id
- 反查"来自哪个文档"
- Week 7 证据链 + OpenLineage 血缘

编排继承

- 图是 Dagster 资产
- 上游变、图自动跟随
- 增量 / 回填 / SLO 全免费

评测继承

- 图增强能进 Week 11 评测
- 按题型分层打分
- 不是孤岛项目

DERIVED ASSET · CODE

把图建成 Dagster 派生资产——编排/血缘/SLO 全部自动继承

看这一段：@asset 一加，实体抽取就成了数据工厂的一等公民

```
# pipelines/assets/graph.py
from dagster import asset, AssetIn, FreshnessPolicy

@asset(
    key_prefix=["support_ops", "gold"],
    name="entity_node",
    group_name="graph_derived",
    ins={"chunks": AssetIn(key_prefix=["support_ops", "silver"])},
    owners=["team:data-platform"],
    freshness_policy=FreshnessPolicy(maximum_lag_minutes=60),
    tags={"criticality": "P2", "asset_type": "derived_graph"},
)
def entity_node(context, chunks):
    """从 chunks 抽实体——复用 Week 6 编排框架"""
    entities = []
    for chunk in chunks:
        result = llm_extract_entities(chunk.text,
                                     response_format=EntityList)
        for e in result.entities: # 自带血缘
            e.source_chunk_id = chunk.chunk_id
            e.source_doc_id = chunk.doc_id
        entities.extend(result.entities)
    return entities
# relation_edge / community / topic_summary 同理，全是 Week 6 资产
```

老司机解读

@asset 让"实体抽取"也成为 Week 6 数据资产 · 编排、回填、血缘全自动来

ins=chunks 显式声明上游 silver 资产 · 上游一变，图自动感知要重算

freshness_policy 让图也有 SLO · 60 分钟内必须刷新，跟其他资产一个待遇

source_chunk_id 注入 · 每个实体都能反查"来自哪个文档"，Week 7 证据链贯穿到图层

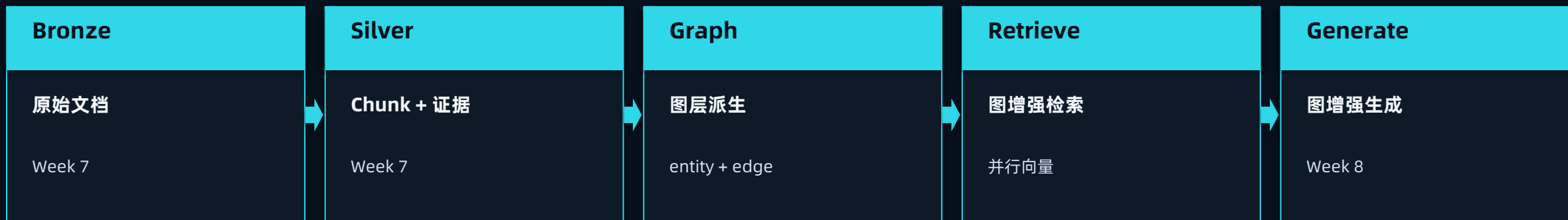
这一刻起，图不再是"独立项目"，它是数据工厂的下游派生层——这句话你记死



ARCHITECTURE

GraphRAG 在数据架构里的位置——派生层，不是独立链

图只是 *silver chunks* 的下游：上游变、图自动跟随，向量索引和图同步演进



关键判断

你把这条链记住：图不是链路的“另一条腿”，是 *silver* 那条腿上长出来的“派生层”。这样一来，Week 1 到 12 攒下的所有工程能力——契约、编排、血缘、评测、可观测——图全部白继承。反过来，你要是把它做成一条独立链路，就等于把前 12 周的功夫在图这儿全部重做一遍，那是纯亏。

INDUSTRY 2026

GraphRAG 的 2026 现状——早不是实验技术，但玩法已经变了

2024 的论文是起点，2026 的落地共识是"轻量化 + 混合 + 按题型路由"

方案 / 平台	定位	2026 工程含义
Microsoft LazyGraphRAG	延迟建图、查询时按需 LLM	索引成本=向量，重写了成本账，首选试水
Microsoft DRIFT Search	Local+Global 融合检索	把两种模式统一，比单独跑更全、更省
Neo4j neo4j-graphrag	官方 Python 包，已 GA	生产落地首选存储 + 检索栈
Graphiti / Zep	双时序知识图（agent 记忆）	解决"实体随时间漂移"，L2 会细讲
Anthropic Contextual Retrieval	给向量 RAG 加上下文	先试它，很多"以为要上图"的场景够用了

老司机说

一句给你省钱的话：别一上来就上重型 GraphRAG。2026 的正确顺序是——先用 Anthropic Contextual Retrieval 把向量 RAG 榨到极限，还不行再上 LazyGraphRAG 试水，真需要复杂关系推理再上完整图 + Neo4j。一步一步来，别一步到位烧钱。

WHEN TO INVEST

上 GraphRAG 前的 5 个判断标准——全过才上

这张表是我给团队定的"上图准入清单"，一项不达标就先别动

判断项	具体问题	阈值	不达标的后果
题型占比	归纳/多跳类占比	> 20%	低于此值收益抵不过成本
数据规模	文档数量	> 1k	太少图结构没意义
关系密度	实体间真有结构关系	高（产品/组织/时序）	低密度图意义不大
查询频率	主场题型 QPS	> 0.1	太低摊不平构建成本
工程能力	有没有 Week 6-12 底子	齐全	没底子上图是空中楼阁

关键判断

用法很简单：5 项全过 → 上 GraphRAG；只过 3 项 → 先做 PoC 跑一轮 A/B 拿数据说话；不到 3 项 → 别折腾，纯向量更稳。注意最后一项，工程能力——这也是为什么这门课把 GraphRAG 放在第 13 周，而不是第 3 周。



LESSON 02 · BUILDING THE GRAPH

从 chunks 到知识图谱：实体抽取 + 关系建模 + 实体对齐

上一讲确认了“该上图”，这一讲讲——怎么从 Week 7 的 chunks 抽出一张真能用的图

真实场景 · 图不是“让 LLM 抽一下”就完了

我第一次带团队做 GraphRAG，图省事，直接拿 LLM 跑了一遍所有 chunk，让它“自由抽实体和关系”。出来的东西我到现在还记得：8 种“产品”类型、12 种“组织”类型、同一家公司在图里有 5 个名字、关系动词五花八门什么都有。这种图，下游的社区检测、路径遍历、子图扩展全被噪声击穿，基本等于废了。我跟你讲——90% 的 GraphRAG 失败，根本不在图算法，就死在构建这一步。图建错，下游再好也救不回来。

咱们这节聊 4 件事：

- 为什么“schema-first”比“让 LLM 自由发挥”靠谱 10 倍——图构建的第一性原理
- 实体抽取的 3 种策略怎么按“精度 × 成本”选——生产默认哪一种
- 实体对齐为什么是最被低估的难点——以及 2026 用时序图怎么解“品牌改名”这种硬骨头
- 把图构建做成 Week 6 数据资产——增量、可回滚、有血缘

SCHEMA FIRST, EXTRACTION SECOND

图构建是工程问题，不是"让 LLM 抽一下"——先定 schema，再做抽取

这一条是 GraphRAG 工程化的第一性原理，记死它

前面说的那个"8 种产品类型、同一家公司 5 个名字"的惨案，根子就一个：没定 schema 就抽。LLM 自由发挥出来的图，类型乱、动词乱、粒度乱，上面所有下游算法都会被噪声击穿。工程化的做法只有一条——先定 schema（哪些节点类型、哪些关系类型），再用"LLM + 规则"约束着抽。

先定 · Node Types

- 节点类型 5-20 种
- Person/Org/Product/Issue
- 类型必须有明确定义
- 不许 LLM 自创类型
- 工程价值：可治理

先定 · Edge Types

- 关系类型 10-30 种
- works_at/produced_by/depends_on
- 动词标准化
- 禁止"related_to"这种含糊
- 工程价值：可推理

先定 · Properties

- 每类节点/边的属性
- name/aliases/domain/source
- 一致命名规范
- 业务必含字段（如时间）
- 工程价值：可查询



REAL SCHEMA

OmniSupport 项目里的图 schema 真实片段

强类型 + 版本号 + 别名 + 血缘——每一个字段都是为了下游可治理

```
# pipelines/graph/schema.yaml
version: 1.0
node_types:
- name: Customer
  properties:
  - {name: customer_id, type: string, required: true}
  - {name: tier, type: enum, values: [free, paid, vip]}
  - {name: aliases, type: list[string]} # ← 实体对齐的基础
- name: Ticket
  properties:
  - {name: ticket_id, type: string, required: true}
  - {name: severity, type: enum, values: [P0, P1, P2, P3]}
  - {name: created_at, type: datetime}
edge_types:
- name: SUBMITTED_BY # 关系名一律大写动词 (Cypher 习惯)
  from: Ticket
  to: Customer
- name: RELATED_TO
  from: Ticket
  to: Issue
  properties:
  - {name: confidence, type: float}
  - {name: source_chunk_id, type: string} # ← 血缘
```

老司机解读

version 字段 · schema 也有版本, Week 14 治理会锁它

type + required · 强 schema 约束, 堵死 LLM 自由发挥

aliases: list[string] · 显式支持别名 (Acme Inc / Acme Corp / Acme), 是实体对齐的地基

source_chunk_id 关系属性 · 每条边都能反查"来自哪个 chunk", Week 7 证据链贯穿到图

关系名一律大写动词 · Neo4j / Cypher 的习惯, 跨工具兼容——细节见功夫

EXTRACTION STRATEGIES

3 种实体抽取策略——按精度 × 成本权衡，抽取是最大成本来源

纯 LLM 精度最高但最贵；纯规则便宜但精度低；混合是工业界事实做法

策略	具体做法	精度	成本/1k chunks	适用
纯规则	NER 模型 + 正则 + 词典	中 70-80%	< \$1	已知实体/简单场景
纯 LLM	让 LLM 自由抽	高 85-92%	\$50-200	复杂关系/灵活领域
LLM + Schema 约束	强制按 schema 输出	高 88-94%	\$30-100	生产推荐默认
混合（规则+LLM 兜底）	规则先扫，复杂 LLM 救	高 85-93%	\$10-40	大规模+成本敏感

关键判断

我的默认配方，给你直接抄：LLM + Schema 约束做主链路，规则给高频实体（客户名、工单号）加速，后面再挂一道 LLM 校验。这套下来精度稳在 90% 上下，成本压在每千 chunk 几十美金——2026 年配 LazyGraphRAG 那套延迟抽取，还能再降一档。

EXTRACTION · CODE

LLM + Schema 约束的实体抽取——用类型系统焊死"不许自创"

看这一段: *Literal* 类型让 LLM 在解码层就只能吐 *schema* 内的东西

```
# pipelines/graph/extract.py
from pydantic import BaseModel
from typing import Literal

class Entity(BaseModel):
    name: str
    type: Literal["Customer", "Ticket", "Product", "Issue"] # 焊死
    aliases: list[str] = []
    source_chunk_id: str

class Relation(BaseModel):
    source: str
    type: Literal["SUBMITTED_BY", "RELATED_TO", "AFFECTS", "RESOLVED_BY"]
    target: str
    confidence: float
    evidence_text: str # 原文片段, 可 review 可溯源

class ExtractionResult(BaseModel):
    entities: list[Entity]
    relations: list[Relation]

def extract_from_chunk(chunk: dict) -> ExtractionResult:
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-2024-08-06", # 结构化输出
        messages=[{"role": "user", "content": PROMPT.format(**chunk)}],
        response_format=ExtractionResult, # 100% 符合 schema
    )
    return resp.choices[0].message.parsed
# Prompt 里硬写: Do NOT invent types / If ambiguous, skip
```

老司机解读

`Literal["..."]` · LLM 在解码层就只能输出 *schema* 内类型, 绝不可能"自创"

`evidence_text` 必填 · 每条关系带原文, 后期能人工 review + 错误溯源

"If ambiguous, skip" · 宁可漏抽不要错抽——错抽进图会污染下游所有推理

`response_format` · pydantic + 结构化输出, 输出 100% 合 *schema*

换 Anthropic 等价写法 · `client.messages.parse(...)` 或 `output_config={"format":{"type":"json_schema"}}`, 课程主线用 `claude-sonnet-4-6` 更省

ENTITY ALIGNMENT

实体对齐——GraphRAG 工程化里最被低估的难点，比抽取本身难 3 倍

抽完你会发现：一个"Acme 公司"在文档里能有 5 种写法

"Acme Inc" / "Acme Corp" / "Acme" / "艾克美" / "ACM"——不对齐，同一个实体在图里就是 5 个节点，关系全被切碎，图基本废了。实体对齐（Entity Resolution）是 GraphRAG 真正的硬骨头，很多团队栽就栽在这。

对齐问题	具体表现	难度	工程解法
命名变体	"Acme Inc"/"Acme Corp"/"Acme"	中	别名词典 + 字符相似度
缩写/翻译	"OpenAI"/"OAI"/"开放 AI"	高	LLM 判断 + 别名集合
类型冲突	同名不同类（产品名 vs 公司名）	高	上下文消歧 + 人工 review
时间漂移	"Twitter" → "X"（品牌改名）	极高	时序图：带时间的别名链（下页）

关键判断

这几类没有银弹，必须组合"别名词典 + 字符相似 + Embedding + LLM + 人工"一起上。尤其最后那行"时间漂移"，2024 年只能靠人工治理，2026 年有了更漂亮的解法——下一页。

TEMPORAL GRAPH · 2026

2026 新解法：用"时序图"根治实体漂移——Graphiti / Zep

"Twitter→X"这种品牌改名，本质是"事实有生命周期"，静态图存不下，时序图能

能力	静态知识图（传统）	时序知识图（Graphiti/Zep 2026）
边的时间	无，只有"当前事实"	每条边带 valid_from / valid_to / invalid_at
"改名前叫啥"	答不了，旧事实被覆盖	原生支持——事实有生命周期，历史可查
生产指标	——	时序推理 +18.5%（LongMemEval）· P95 300ms

老司机解读

这套思路是这两年 agent 记忆方向卷出来的——Zep 那篇 Graphiti 论文把"双时序"做成了图的一等公民：一条边不光记"A 拥有 B"，还记"这个事实从什么时候到什么时候是真的"。于是"Twitter 2023 年改名 X"不再是让你头疼的对齐冲突，而是别名链上一个带时间戳的正常事件——你甚至能回答"改名前它叫什么"。

落地判断：如果你的领域实体会随时间变（公司改名、并购、人员变动、合同版本），别再用静态图硬扛对齐了，2026 直接上 Graphiti 这类时序图，把"漂移"从 bug 变成 feature。

ALIGNMENT · CODE

实体对齐的 4 阶段分层——让 95% 免费完成，只有 5% 走 LLM

大规模图构建必须这么分层，不然对齐成本比抽取还爆

```
# pipelines/graph/align.py
from rapidfuzz import fuzz

def resolve_entity(new: Entity, existing: list[Entity]) -> Entity | None:
    # 阶段 1 · 别名词典（最快，命中率最高，处理 70% 高频）
    for e in existing:
        if new.name == e.name or new.name in e.aliaes:
            return e

    # 阶段 2 · 字符相似度（拼写/大小写/标点差异，rapidfuzz 极快）
    cand = [(e, fuzz.ratio(new.name, e.name)) for e in existing
            if e.type == new.type]
    cand = [c for c in cand if c[1] > 85]
    if cand and max(c[1] for c in cand) > 92:
        return max(cand, key=lambda c: c[1])[0]

    # 阶段 3 · Embedding 相似度（同义词/翻译变体，中等成本）
    if cand:
        ne = embed(new.name)
        for e, _ in cand:
            if cosine(ne, embed(e.name)) > 0.92:
                return e

    # 阶段 4 · LLM 兜底（上下文模糊，贵但量小）
    if cand and llm_judge_alignment(new, cand).merge:
        return llm_judge_alignment(new, cand).target
    return None  # 新建节点
```

老司机解读

阶段 1 词典 · 处理 70% 高频实体，几乎零成本

阶段 2 字符相似 · 拼写/大小写/标点，rapidfuzz 快到可忽略

阶段 3 embedding · 同义词/翻译变体，中等成本

阶段 4 LLM · 只兜底上下文模糊的，贵但量小

这套分层让 95% 的对齐免费完成，只有 5% 掏 LLM 的钱——大规模图构建不这么分层，光对齐就能把预算烧穿

INDUSTRY 2026

图构建工具生态——2026 选型

从 0 起步别自己造轮子，抽取 + 对齐 + 存储都有成熟栈

工具	定位	2026 建议
Microsoft GraphRAG / LazyGraphRAG	端到端框架	从 0 起步 + 想省成本，首选 Lazy 版
Neo4j neo4j-graphrag (GA)	官方 Python 抽取+存储	生产级、可视化（自建集群约 \$300+/月）
LangChain LLMGraphTransformer	把"抽实体关系"做成组件	配 schema 约束，快速 PoC
Graphiti / Zep	时序知识图	实体随时间漂移的领域必看
Diffbot Knowledge Graph	商业预构建图	金融等不想自建团队

老司机说

我的选型路径：PoC 用 Microsoft GraphRAG 快速验证 → 生产存储上 Neo4j（neo4j-graphrag 包已 GA，别自己写 Cypher 胶水）→ 实体会随时间变的领域，加 Graphiti 那套时序层。别在"造轮子"上浪费三个月，这些栈都够成熟了。

LESSON 03 · GRAPH-ENHANCED RETRIEVAL

图增强检索：子图扩展、社区检测、路径遍历——让"关系"也能被召回

Lesson 2 把图建好了，这一讲讲——怎么用它做检索

真实场景 · 把图当向量库用，等于白建

我 review 过一个团队的 GraphRAG，图建得挺漂亮，可一看检索代码我就乐了——他们还是"查相似 chunk"那一套，只不过多召回了几个图节点。这么用，图的钱全白花了。图真正的价值，是三种向量根本做不到的检索模式：子图扩展（Local）、社区检测（Global）、路径遍历（Multi-hop）。每一种，解一类向量做不到的题。这一讲我把三种模式讲透，再告诉你 2026 微软怎么把前两种拧成一个 DRIFT Search。

咱们这节聊 4 件事：

图检索的核心不是"查得更准"——是"召回向量召回不回来的东西"

Local / Global / Multi-hop 三种模式各解一类题，怎么按题型路由

Global Search 的社区检测 + Map-Reduce——微软论文的核心创新，以及 2026 的 DRIFT 融合

Multi-hop 为什么最容易图爆炸——4 道护栏一道都不能少



RECALL WHAT VECTOR CANNOT

图检索的核心——召回向量召回不回来的东西

两种检索目标根本不同：向量找“长得像答案”，图找“通过关系连到答案”

向量检索

- 基于“语义相似”
- 找“长得像答案”的 chunk
- “重置密码”→ 找操作步骤
- 答案在单个 chunk 内
- 这是它的舒适区

图检索

- 基于“结构关系”
- 找“通过关系链到答案”的实体
- “半年 P0 共性”→ 找一组关联实体
- 答案在实体之间
- 向量的盲区

工程结论

- 两种并存，不是二选一
- 按题型路由到不同检索
- 继承 Week 8 的 intent routing
- 图 + 向量混合是标配
- 单一模式都不够用

关键判断

你把这句刻在脑子里：图检索的工程价值 = 召回向量召回不回来的东西。所以生产里从来不是“图取代向量”，而是两种检索并存、按题型路由——这也是为什么下一节我要专门讲一个“题型分类器”。



THREE SEARCH MODES

图检索的 3 种核心模式——各解一类向量做不到的题

Local 聚焦、Global 归纳、Multi-hop 遍历，代价从低到高

LOCAL · 子图扩展

- 以一个实体为起点召回邻居
- 解决：聚焦类问题
- "客户 X 的所有工单和问题"
- 查询代价：低
- 最常用

GLOBAL · 社区归纳

- 社区检测 + Map-Reduce
- 解决：全局归纳类
- "过去 6 个月主要故障类型"
- 查询代价：高（需预计算）
- 微软论文核心

MULTI-HOP · 路径遍历

- 从一实体走 $\leq N$ 跳
- 解决：关系遍历类
- "A 公司的子公司的供应商"
- 查询代价：中（限跳数）
- 最容易爆炸

老司机说

别指望一种模式打天下。"客户 X 的所有工单"是 Local，"故障类型分布"是 Global，"A 的子公司的供应商"是 Multi-hop——同一套图，三种问法要走三条完全不同的检索路。选错模式，图召回再全也答不对题。

MODE SELECTION

3 种模式的选择标准——按题型路由，4 选 1

这个路由由"题型分类器"做，Week 8 学过的 intent routing 在这儿继续用

题型	示例	检索模式	为什么
聚焦实体类	"客户 X 的所有工单"	Local Search	已知锚点 + 邻居扩展
全局归纳类	"P0 故障类型分布"	Global Search	需要全图聚合
关系遍历类	"A 公司收购了谁"	Multi-Hop	需要走结构关系
具体事实类	"重置密码步骤"	向量 RAG	图用不上，别浪费
对比类	"V1 和 V2 的差别"	Local + 向量混合	结构 + 内容都要

关键判断

注意最后两行——不是所有题都非得走图。"重置密码"老老实实走向量，"V1 vs V2"要图 + 向量混着来。好的 GraphRAG，一半功夫花在"什么时候别用图"上。

LOCAL SEARCH · CODE

Local Search——子图扩展实战（Neo4j + APOC）

从锚点实体出发扩展 1-2 跳，限住 *maxLevel* 防图爆炸

```
# services/graph/local_search.py
def search(self, query: str, max_hops: int = 2) -> dict:
    # 1. 识别 query 里的锚点实体（NER + L2 的实体对齐）
    anchors = self._extract_anchors(query)
    if not anchors:
        return {"error": "no_anchor", "fallback": "vector"} # 降级

    # 2. Cypher: 从锚点 BFS 扩展 max_hops 跳
    result = session.run("""
        MATCH (a) WHERE a.name IN $names
        CALL apoc.path.subgraphAll(a, {
            maxLevel: $h,
            relationshipFilter: "SUBMITTED_BY|RELATED_TO|AFFECTS",
            bfs: true
        }) YIELD nodes, relationships
        RETURN nodes, relationships
    """, names=anchors, h=max_hops).single()

    # 3. 通过 source_chunk_id 反查原文 chunks
    chunk_ids = {r["source_chunk_id"] for r in result["relationships"]}
    chunks = chunk_store.batch_get(chunk_ids)

    # 4. "子图 + 原文" 一起交给生成层
    return {"mode": "local_search", "anchors": anchors,
            "subgraph": result, "evidence_chunks": chunks}
```

老司机解读

apoc.path.subgraphAll · Neo4j 标准 BFS 扩展，maxLevel 限死防图爆炸——别自己写遍历

relationshipFilter · 只走业务相关关系，不让"任意边"污染子图

source_chunk_id · 子图反查原文，生成层既有结构又有证据

fallback="vector" · 识别不出锚点（"如何重置密码"）就降级走向量，用户无感

这就是 Local 最实用的地方——它对"降级"很友好，是最先该上线的模式



GLOBAL SEARCH · FLOW

Global Search——社区检测 + Map-Reduce，微软论文的核心创新

把“看全图”拆成“看 N 个社区再合并”，复杂度 $O(N^2)$ 降到 $O(N)$



关键判断

这一步的精髓，是“预计算”——社区检测和摘要不是每次查询现算，是建完图批量算一次、入库存着。用户问“全局类问题”时，只做 Map（并行读各社区摘要）+ Reduce（LLM 汇总）。这就是为什么 Global Search 建图贵、但查询能扛——重活提前干完了。这也是 2026 LazyGraphRAG 要优化的正是这块的成本。

COMMUNITY DETECTION · CODE

社区检测 + 摘要预计算——Leiden 是工业界事实选择

比 NetworkX 的 Louvain 快 10 倍、更稳定；摘要预计算产物

```
# pipelines/graph/community.py
import leidenalg as la
import igraph as ig

def detect_communities(graph_data: dict) -> list[dict]:
    g = ig.Graph()          # igraph 性能好
    g.add_vertices([n["id"] for n in graph_data["nodes"]])
    g.add_edges([(e["src"], e["tgt"]) for e in graph_data["edges"]])

    # Leiden: 比 Louvain 更稳定, 不会产生"断裂社区"
    partition = la.find_partition(g, la.ModularityVertexPartition)

    communities = []
    for cid, members in enumerate(partition):
        chunk_ids = set()
        for idx in members:
            chunk_ids.update(graph_data["nodes"][idx]["source_chunks"])
        # LLM 生成社区摘要 (预计算, 不是查询时算)
        summary = llm_summarize(
            chunks=chunk_store.batch_get(chunk_ids),
            instruction="Summarize as: topic + key facts + key entities")
        communities.append({"id": cid, "size": len(members),
                           "summary": summary,
                           "source_chunks": list(chunk_ids)})
    return communities
# 建图后跑一次; 进 Week 14 release manifest; 每周/大批量入图时重跑
```

老司机解读

igraph + Leiden · 工业界事实选择, 比 NetworkX Louvain 快 10 倍、更稳

summary 是"预计算"产物 · 建图后批量算并入库, 不是每次查询现算

source_chunks 保留 · Map 阶段可同时返回摘要和具体原文

社区检测要进 release manifest · 图变→社区可能变→摘要要重算, Week 14 治理的活

2026 提醒 · LazyGraphRAG 把"给每个社区都预生成摘要"这步的钱省了——延迟到查询时按需做

DRIFT SEARCH · 2026

2026 别再把 Local / Global 当两条路——微软 DRIFT 把它俩融合了

DRIFT = Dynamic Reasoning and Inference with Flexible Traversal

维度	Local / Global 分开跑（2024）	DRIFT Search（2024 末起）
起点	Local 只从锚点，Global 只看社区	Local 起点里就注入社区信息，起点更宽
覆盖	选错模式就漏答案	一次检索兼顾"聚焦细节 + 全局背景"
效果	两种各有盲区	召回的事实种类明显更多、答案更全

老司机解读

DRIFT 的思路很实在——用户问一个聚焦问题时，光看锚点周围往往不够，它把"这个锚点属于哪个社区、社区的大背景是什么"一起带进检索起点，然后动态地在局部和全局之间来回走。相当于把 Local 的"细"和 Global 的"广"揉进了一次查询。

给你的落地建议：新项目别再吭哧吭哧手写 Local / Global 两套路由再纠结怎么合并了，直接用微软 GraphRAG 里的 DRIFT 模式起步，它已经把"该看局部还是全局"这个判断内化掉了。这是 2024 论文那套"二选一"范式在 2026 的直接升级。

MULTI-HOP TRAPS

Multi-Hop 的 4 个坑——图最强的能力，也是最容易爆炸的能力

一道护栏都不能少，少一道就翻车

坑	具体表现	后果	解法
无跳数限制	"找所有相关"无上限	召回 1000+ 节点	$\text{max_hops} \leq 3$ ，常用 2
关系不过滤	走所有边	噪声 + 语义偏离	relationshipFilter 白名单
不打分	BFS 平等对待	强弱关系混一起	按距离 + 边权重打分
不验证路径	"有路径就相关"	语义可能不连贯	LLM 验证路径语义

踩坑提醒

我踩过最狠的一次：一个没限跳数的 multi-hop 查询，在一张 5 万节点的图上跑，一口气召回 8000 多个节点，直接把 LLM 上下文撑爆、把这次请求的成本干到平时的 40 倍。Multi-hop 是图最性感的能力，但你不给它上四道护栏，它就是个吞钱吞算力的黑洞。

MULTI-HOP · CODE

受控 Multi-Hop——四道护栏焊在一段 Cypher 里

限跳数 + 关系白名单 + 路径打分 + LLM 验证, 一个都不少

```
# services/graph/multihop.py
def multihop_search(start, target_pattern, max_hops=2,
                    allowed_edges=None) -> list[dict]:
    if max_hops > 3:
        raise ValueError("max_hops too large, will explode") # 护栏1
    query = f"""
    MATCH path = (s {{name: $start}})
    -[r:{"|".join(allowed_edges)}*1..{max_hops}]->(t) // 护栏2 白名单
    WHERE t.name CONTAINS $pat OR t.type IN $types
    WITH path,
        reduce(sc = 1.0, e in r | sc * e.confidence) AS score // 护栏3 打分
    WHERE score > 0.5
    RETURN path, length(path) AS hops, score
    ORDER BY score DESC, hops ASC
    LIMIT 20
    """
    paths = list(session.run(query, ...))
    # 护栏4: LLM 看一眼路径逻辑通不通 (防"机械正确、语义无关")
    return [p for p in paths if llm_verify_path_coherence(p["path"])]
```

老司机解读

max_hops ≤ 3 硬约束 · 4+ 跳几乎没法解释, 先把图爆炸堵死

relationshipFilter 白名单 · 只走业务关系, 不让无关边混入

score 累乘 confidence · 强关系优先, 把"图的稳定边"和"LLM 抽的边"区分开

LLM 验证连贯性 · 路径 score 高不代表逻辑通, 让 LLM 兜最后一道

2026 备选 · HippoRAG 2 用 personalized PageRank 一步做多跳, 省掉迭代遍历, 适合关系很密的图

INDUSTRY 2026

图检索的工业界标杆——别自己造轮子

Local + Global + 向量混合是共识，2026 又多了 DRIFT 和单步多跳

方案	能力	2026 建议
Microsoft GraphRAG (DRIFT)	Local/Global/DRIFT 融合	新项目直接用 DRIFT 起步
Neo4j + APOC	subgraphAll / Dijkstra / 社区检测	开箱即用，别手写图算法
HippoRAG 2	personalized PageRank 单步多跳	关系密、多跳深的图值得一试
LangChain GraphCypherQChain	自然语言转 Cypher	复杂关系探索，让 LLM 写查询
LlamaIndex KG Retriever	图 + 向量混合默认	上手成本低，快速验证

老司机说

一句话选型：存储和图算法交给 Neo4j + APOC，检索范式跟微软 GraphRAG 的 DRIFT 走，真遇到深多跳、关系密的图再评估 HippoRAG 2。你要写的，只有"题型分类器"和"prompt 模板"这两块业务胶水——其他全是现成的。



LESSON 04 · GRAPH-AUGMENTED GENERATION

图增强生成：怎么把"结构化的图"塞进 LLM 的 prompt

Lesson 3 把图数据召回了——但 LLM 不会自然消费"节点 + 边"这种结构

真实场景 · 图召回得再好，喂法不对照样翻车

有次我们的 *Global Search* 召回质量明明很高，可最终答案还是稀烂。查了半天，问题出在最后一公里——工程师图省事，直接把召回的图 JSON（一堆 *node*、*edge* 的字典）原封不动塞进了 *prompt*。LLM 拿到那坨 JSON，给你一串似是而非的解读。这不怪模型——LLM 是在文本上训练的，你喂它"A 收购了 B"这句话，它秒懂；你喂它 `{"src": "A", "edge": "acquire", "tgt": "B"}`，它就懵。这一讲讲清楚：怎么把图翻译成 LLM 吃得香的文本。

咱们这节聊 4 件事：

- 为什么"直接把图 JSON 丢进 prompt"几乎一定让生成更差——LLM 不会读图
- 3 种序列化策略（路径文本 / 子图表格 / 社区摘要）对应 3 种检索模式
- 为什么 Local / Global / Multi-hop 必须各写一份 prompt 模板，不能一套通用
- 图增强怎么不丢 Week 8 的证据链——反而要把 evidence 扩展成"双链"



LLM CANNOT READ GRAPHS

LLM 不会读图——必须先把图"翻译成它擅长的文本"

图增强生成的核心工程问题，不是图算法，是"图 → 文本"的翻译质量

路径 → 文本

- 多跳路径写成自然语言
- "A 收购 B; B 拥有 X"
- 配合: Multi-Hop
- 优点: LLM 友好
- 局限: 长路径冗长

子图 → 表格

- 子图变 markdown 表
- 行: 节点 / 关系 / 邻居
- 配合: Local Search
- 优点: 结构清晰
- 局限: 吃上下文

社区 → 摘要

- 预计算的社区摘要
- "主题 + 关键事实 + 实体"
- 配合: Global Search
- 优点: 信息密度高
- 局限: 丢局部细节

关键判断

一个原则你记死：能用文本就别用 JSON。LLM 在文本上训练，文本永远比结构化 JSON 好懂。实测里，同一个子图，用 markdown 表喂比用 JSON 喂，答案准确率能差 30% 上下——这一步的翻译，值得你花心思。



SERIALIZATION · CODE

3 种序列化的实现——都保留 chunk 血缘

路径走成句子、子图变成表、社区做成摘要块

```
# services/graph/serialize.py
# 1. Path → Text (for Multi-Hop)
def serialize_path(path: list[dict]) -> str:
    parts = [f"{e['source']]['name']} "
             f"{e['relation'].lower().replace('_', ' ')} "
             f"{e['target']]['name']" for e in path]
    return ". ".join(parts) + "." # "Acme acquired Beta. Beta produces X."

# 2. Subgraph → Markdown Table (for Local Search)
def serialize_subgraph(sg: dict) -> str:
    lines = [f"# Subgraph on: {sg['anchor']}", "",
            "| Entity | Type | Relation | Connected To |",
            "|---|---|---|---|"]
    for e in sg["edges"]:
        s_, t_ = sg["nodes"][e["src"]], sg["nodes"][e["tgt"]]
        lines.append(f"| {s_['name']} | {s_['type']} | {e['type']} | {t_['name']} |")
    return "\n".join(lines)

# 3. Community → Summary Block (for Global Search)
def serialize_community(c: dict) -> str:
    return (f"Community {c['id']} ({c['size']} entities)\n"
           f"Topic: {c['summary']['topic']}\n"
           f"Key Facts: {'; '.join(c['summary']['key_facts'])}\n"
           f"Source: {len(c['source_chunks'])} docs")
```

老司机解读

serialize_path · 把 (src)-[rel]-(tgt) 变成 "X 动词 Y", LLM 立刻能懂

serialize_subgraph · markdown 表, GPT/Claude 对 table 解读非常稳, 比 JSON 准约 30%

serialize_community · 预计算摘要模板化, Global 用它, token 效率最高

三种都保留 chunk_id / source · Week 7 证据链贯穿到图增强生成

一个原则 · 能用文本就别用 JSON——这是图增强的第一手艺



PROMPT TEMPLATES

3 种检索模式对应 3 套 prompt 模板——不能一套通用

Local 要"基于具体实体说", Global 要"跨社区归纳", 写混了召回再好也白搭

```
# prompts/graph_local.j2
Context (subgraph centered on {{ anchor }}):
{{ serialized_subgraph }}
Evidence chunks:
{% for c in chunks %}- [{{ c.chunk_id }}] [{{ c.text[:300] }}]{% endfor %}
Question: {{ query }}
Rules: 只用于子图内实体 + 证据 chunk; 不要发明未展示的关系; 用 [chunk_id] 引用
----

# prompts/graph_global.j2
Summaries of {{ communities|length }} relevant communities:
{% for c in communities %}## Community [{{ c.id }}]
[{{ c.summary }}]{% endfor %}
Question: {{ query }}
Rules: 跨社区综合出全局规律; 识别共性 / 异常; 引用社区 ("Community 3 显示...")
----

# prompts/graph_multihop.j2
Relationship paths (sorted by relevance):
{% for p in paths %}- [{{ p.serialized_path }}]{% endfor %}
Question: {{ query }}
Rules: 清晰追踪关系链 "A → B → C"; 每步必须有上面路径支撑; 无路径就直说答不了
```

老司机解读

graph_local · 强调"只用于子图 + 不要发明关系", 防 LLM 在熟悉领域里编造

graph_global · "跨社区综合规律", 这正是 Global 的目标

graph_multihop · 要求把路径用 → 链表示, 让答案可被审计

3 个模板都强制 citation · Week 7-8 证据链在图层延续

Jinja2 + Week 8 Prompt as Code · 改模板就能改图增强行为, 还能版本化、回滚



GRAPH EVIDENCE

图增强的 evidence——扩展 Week 8 契约，做成"双链"

图增强后必须保留 Week 8 的 evidence 契约，但要从"只有 chunk"扩展到"chunk + 图"

```
# services/graph/response.py
class ChunkEvidence(BaseModel):    # Week 8 原样保留，客户端不用改
    chunk_id: str; doc_id: str; page_no: int; text: str

class GraphEvidence(BaseModel):    # 图增强新增
    type: Literal["subgraph", "path", "community"]
    anchor_entity: Optional[str] = None
    path_text: Optional[str] = None
    community_summary: Optional[str] = None
    source_chunks: list[ChunkEvidence] = [] # 图证据也能反查原文

class RAGResponse(BaseModel):
    answer: str
    chunk_evidences: list[ChunkEvidence] = []
    graph_evidences: list[GraphEvidence] = [] # ← 新增
    retrieval_mode: Literal["vector", "local_graph",
        "global_graph", "multi_hop", "hybrid"] # ← 让评测能分模式
    confidence: float
    trace_id: str
    release_id: str
```

老司机解读

ChunkEvidence 不变 · 完全兼容 Week 8 纯向量 RAG，客户端代码零改动

GraphEvidence 新增 · 前端能可视化"图证据"：渲染子图、显示路径、展示社区

source_chunks 在图证据里也保留 · 图证据能反查原文，双层证据链

retrieval_mode 显式 · 审计/评测知道这次用图还是向量，Week 11 能分模式统计

这个字段是下一节 A/B 的地基——没有它，你永远算不清"图到底在哪类题上赢了"



THE FULL PIPELINE

图增强生成的完整流水线——5 步走完，90% 代码复用

完全继承 Week 7-8 工程基础，只在序列化和 evidence 两层做图扩展



关键判断

你数数这五步里有几步是新的——只有 **Serialize（图→文本）** 和 **Cite 里的 GraphEvidence** 是图增强真正新增的。Route 是 Week 8 的 intent routing，Retrieve 是 Lesson 3，Generate 是 Week 8 的受约束生成模板。所以我一直说：图增强不是另起炉灶，是在你已有的 RAG 流水线上插两块，90% 的代码原样复用。

COMMON PITFALLS

图增强生成的 5 个反模式——每一个我都见团队栽过

精髓就一句：让 LLM 用它最擅长的方式（文本）消费图数据

反模式	具体表现	后果	正确做法
直接 JSON 喂	图节点 dict 丢 prompt	LLM 解读差	序列化成 path/table/summary
一套模板通用	Local/Global 同一 prompt	归纳变枚举、聚焦变发散	3 套模板
丢 chunk 证据	只给图不给原文	客户问"凭证"答不上	source_chunks 必带
mode 不暴露	图和向量结果混着	评测无法分模式	retrieval_mode 必填
路径太长	multi-hop 序列化 7+ 跳	LLM 失焦	max_hops ≤ 3

踩坑提醒

第三条"丢 chunk 证据"我要特别点一下——图增强很容易让人只顾着炫"图关系"，把原文引用丢了。结果客户一句"你凭什么这么说"就把你问倒。记住：图给的是"结构/线索"，原文给的是"证据"，在合规和信任上，证据永远比线索重要。双链一个都不能少。

INDUSTRY 2026

图增强生成的行业实践——核心不是图算法，是序列化 + 模板

这一层是纯工程活，各家最佳实践高度一致

来源	做法	给你的启示
Microsoft GraphRAG	Local prompt + Global Map-Reduce	两套模板是官方范式，直接抄
Anthropic Cookbook	"path → 自然语言"最佳实践	关系写成句子，LLM 才真懂
LangChain GraphRetriever	检索→序列化→生成链式 API	开发者只关心模板
LlamaIndex KGRetriever	默认 chunk + graph 混合 evidence	前端同时渲染原文和图，信任高

老司机说

这一节其实没什么高深算法，全是"手艺"——图怎么翻译成文本、模板怎么写、证据怎么带。但恰恰是这些手艺，决定了你前面辛辛苦苦建的图、调的检索，最后能不能变成一个好答案。我见过太多团队图建得漂亮、检索也准，就栽在这最后一公里的翻译上。这一层，是最不该省功夫的地方。

LESSON 05 · A/B + ADOPTION DECISION

A/B 对比 + 适用边界：用真实数据决定"GraphRAG 上还是不上"

前 4 节把 GraphRAG 完整做出来了——这一讲讲怎么用数据证明它值不值

真实场景 · 上线半年，没人能回答"这玩意儿到底有没有用"

我见过最典型的 GraphRAG 失败，不是技术失败——是上线半年后，业务方问"我们花这么多钱上的图，效果到底怎么样"，团队支支吾吾拿不出数据。为什么拿不出？因为当初只看了个总体指标"F 从 0.85 涨到 0.87"就上线了，从来没按题型分开看。而那 2 个点很可能是：70% 简单题图还差一点，30% 归纳题图好一大截，一平均，略涨。这一讲讲清楚：A/B 怎么按题型分层做、怎么把 GraphRAG 变成"可量化决策"，而不是一个说不清的 buzzword。

咱们这节聊 4 件事：

为什么"看总体指标"会骗你——必须按题型分层看效果

GraphRAG vs Vector 的 A/B 框架 4 要素——并行/分层/多模型 Judge/影子流量

OmniSupport 200 样本的真实 A/B 数据——看完你就懂"为什么必须按题型路由"

怎么把 GraphRAG 接入 Week 11/12/14——不做接入，它半年后就是沉没成本

NOT "DOES IT WORK"

别问"GraphRAG 成不成功"——问"在哪些题型上成功、在哪些其实没必要"

初次上图最容易犯的错：被"总体指标略涨"骗着上线

看总体指标——" $F=0.87$ vs 0.85 ，涨了 2 个点，上线吧！"——这是我见过最常见的误判。这 2 个点很可能是这么来的：70% 简单题型，图比向量差一点；30% 归纳类题型，图比向量好很多；加权平均，略涨。真正的价值判断应该是：那 30% 归纳类涨了多少 vs 多花的 5-10 倍成本——回得来本吗？

错误的看法

- 看总体加权平均
- "涨了 2 个点，上"
- 掩盖了局部翻车/翻盘
- 成本被平均稀释
- 拍脑袋决策

正确的看法

- 按题型分层看
- 每类题单独算账
- 归纳题涨多少、成本多少
- 主场题型占比够不够
- 数据驱动决策

决策公式

- 主场题型收益 $\times$ 占比
- $-$ (图成本 $-$ 向量成本)
- > 0 才上
- 让数据告诉你
- 不是凭感觉

A/B FRAMEWORK

GraphRAG A/B 实验的 4 个核心设计——缺一不可结论都不可信

这是工业界判断 GraphRAG 是否上线的科学标准

设计项	具体做法	为什么必须
并行运行	同 query 同时跑 Vector / Graph 两路	消除时间偏差
按题型分层	4 类（局部/全局/多跳/事实）分别统计	避免加权平均误导
多模型 Judge	GPT + Claude 互判（Week 11 交叉评审）	消除单一 judge 的偏见
真实流量影子	5-10% 真实流量复制到 Graph	收集真实分布数据，不靠造样本

关键判断

这里我要强调"影子流量"——别拿你自己造的 100 条评测题下结论，那跟真实用户问法差很远。把 5-10% 的真实线上流量镜像一份给 Graph 跑（只评测、不返回给用户），一两周下来，你拿到的才是真实题型分布。很多团队就是死在"实验室数据好看、上线拉胯"这个坑里。

A/B · CODE

GraphRAG vs Vector 的 A/B——并行两路 + 按题型分层统计

核心就一个 *by_category*，别让全局平均掩盖局部真相

```
# evals/graphrag_ab.py
CATEGORIES = ["focal", "global", "multihop", "factual"]

def run_ab(eval_set: list[dict]) -> dict:
    results = []
    with ThreadPoolExecutor(max_workers=8) as ex:
        for s in eval_set:
            vec = ex.submit(vector_rag_query, s["query"]).result()
            graph = ex.submit(graphrag_query, s["query"]).result() # 并行
            results.append({
                "category": s["category"],
                "vec_score": ragas_evaluate(s, vec), # Week 11 交叉评审
                "graph_score": ragas_evaluate(s, graph),
                "vec_cost": vec["cost_usd"],
                "graph_cost": graph["cost_usd"],
            })
    # 按 category 分层 —— 这是核心
    report = {}
    for cat in CATEGORIES:
        rows = [r for r in results if r["category"] == cat]
        report[cat] = {
            "n": len(rows),
            "vec_f": mean(r["vec_score"]["faithfulness"] for r in rows),
            "graph_f": mean(r["graph_score"]["faithfulness"] for r in rows),
            "cost_overhead": mean((r["graph_cost"] - r["vec_cost"])
                                  / r["vec_cost"] for r in rows),
        }
    return report # 跑进 Week 11 PR Gate, 每次变更自动评
```

老司机解读

ThreadPoolExecutor 并行 · 两路同上下文、同时间，消除"时间漂移"偏差

ragas_evaluate 交叉评审 · GPT + Claude 互判，消除 judge bias (Week 11)

by_category 分层 · 核心——避免"全局平均"掩盖局部翻车/翻盘

cost_overhead · 关键决策指标，Faithfulness +5% 但成本 +500% 不一定划算

跑进 Week 11 PR Gate · 任何 graph 模型/prompt 变更都自动评测，不靠人工记得

TYPICAL A/B RESULTS

OmniSupport 的真实 A/B 数据（200 样本）——一张表说清一切

这张表我建议你贴给业务方看——比任何 PPT 都有说服力

题型	样本数	Vec 忠实度	Graph 忠实度	成本倍数	推荐
局部聚焦	70 (35%)	0.91	0.88	+450%	Vector 胜（图无意义）
全局归纳	40 (20%)	0.52	0.83	+380%	Graph 胜（必须图）
多跳推理	35 (17.5%)	0.61	0.84	+520%	Graph 胜（必须图）
具体事实	55 (27.5%)	0.93	0.90	+400%	Vector 胜（图无意义）

关键判断

结论一目了然：归纳 + 多跳（37.5%）必须上图，忠实度从 0.5 出头拉到 0.83 以上，这是向量根本够不着的；局部 + 事实（62.5%）纯向量更划算，图不光没提升还多花 4-5 倍成本。所以 GraphRAG 唯一正确的用法就是——按题型路由，让图只在它的 37.5% 主场上场。

RETRIEVAL ROUTING

按题型自动路由——图只在主场上场，客场让位给向量

这条路由链让 GraphRAG 收益最大化：主场提质、客场省钱



关键判断

这条路由链是 GraphRAG 落地的"总开关"。它让图在归纳/多跳这 37.5% 的主场题型上发力提质，在事实/聚焦这 62.5% 的客场题型上自动让位给便宜的向量——收益最大、成本最小。而且分类器用最便宜的小模型就行，是整条 GraphRAG 流水线里成本最低的一环。

QUERY CLASSIFIER

题型分类器——用最便宜的小模型，是整条流水线成本最低的一环

结构化输出焊死 4 类，低置信度自动降级走向量

```
# services/graph/classifier.py
class QueryClassification(BaseModel):
    category: Literal["focal", "global", "multihop", "factual"]
    confidence: float
    suggested_retrieval: Literal[
        "vector", "local_graph", "global_graph", "multi_hop"]

def classify(query: str) -> QueryClassification:
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-mini",      # 用 mini，比大模型便宜 ~30 倍
        messages=[{"role": "user",
                    "content": CLASSIFIER_PROMPT.format(query=query)}],
        response_format=QueryClassification,
    )
    return resp.choices[0].message.parsed

# 在 RAG service 中:
c = classify(query)
mode = c.suggested_retrieval if c.confidence > 0.7 else "vector" # 兜底
```

老司机解读

gpt-4o-mini 做分类 · 比大模型便宜约 30 倍，分类一条 query 约 \$0.0003——整条流水线最便宜的一环

换 Anthropic · claude-haiku-4-5 更省，课程主线用它

结构化输出 · 永远输出 4 类之一，不会乱写

confidence < 0.7 降级走向量 · 防止误分类把简单题推到贵的图链路

"轻量 LLM 分类 + 重量级主链路处理" · 2026 Agent 工程标配，省钱又稳

INTEGRATION CHECKLIST

GraphRAG 接入已有工程的 5 类必做——从上线第 1 天就做

不做接入，GraphRAG 半年后就是没人维护、没法定位的沉没成本

工程能力	具体集成	来源	不做的后果
评测	题型分层评测 + golden set	Week 11	没法判断图是否劣化
可观测	OTel span（Phoenix/Langfuse）覆盖图检索 + 序列化	Week 12	出问题不知哪段慢
告警	Graph 服务 SLO + 错误预算	Week 12 L4	Graph 挂了客户先发现
治理	release manifest 含 graph_release	Week 8 + Week 14	图版本和向量版本不对齐
Bad case	Graph 特有反例（错路径/错社区）	Week 12 L5	同类错反复发生

踩坑提醒

这 5 类接入，必须从上线第 1 天就做，别信“以后再加”这种话——加在已经跑起来的 service 上，比从 0 建难 10 倍。我见过太多 GraphRAG 项目，就是因为上线时图当外挂、没接评测和可观测，半年后成了没人敢碰的黑盒，最后只能整个下线重来。

RELEASE MANIFEST

GraphRAG 纳入 release manifest——图是 release 里的一等公民

继承 Week 8/9/11: 图的所有元素原子绑定、可回滚

```
# release/manifests/rag-v2026.05.18-001.yaml
apiVersion: omnisupport.rag/v4      # ← v4 加 graph_release
spec:
  # ...已有 index / prompt / model / skill_pack / eval / business_slo
  graph_release:                     # Week 13 图层也必须绑
    schema_version: 1.2.0
    extractor_model: gpt-4o-2024-08-06
    entity_count: 18,420
    edge_count: 47,830
    community_count: 142
    community_summary_version: v1.5
    last_full_rebuild: 2026-05-18T03:00:00Z
  query_classifier: {prompt_version: cls-v1.0, confidence_threshold: 0.7}
  eval:                             # 按题型分层的基线, 锁住
    by_category:
      global: {vec_f: 0.52, graph_f: 0.83, mode: global_graph}
      multihop: {vec_f: 0.61, graph_f: 0.84, mode: multi_hop}
  rollout:
    auto_rollback_on:               # 图专项回滚条件
      - {metric: graph_extraction_failure_rate, threshold: 0.05}
      - {metric: graph_query_p99_ms, threshold: 8000}
```

老司机解读

graph_release 段 · 把图的所有元素 (schema/抽取器/数量/社区/摘要) 原子绑定, 锁版本

query_classifier 也纳入 · 改分类器 = 改 release

eval.by_category · 按题型分层的基线锁住, 下次评测必须分别看每一类, 不许总体平均掩盖

auto_rollback_on 加图专项 · 抽取失败率/图查询延迟超阈值自动回滚

这一刻起 · GraphRAG 是 release 体系的一等公民, 跟索引/Prompt/模型同等治理

INDUSTRY 2026

GraphRAG 的 2026 现状——特定场景的工业刚需，但要诚实

一边是真金白银的降本，一边是“很多场景其实不需要图”的清醒

来源	信号	给你的判断
Bloomberg / Goldman / LexisNexis	图+向量混合、按题型路由，年省成本 40-60%	金融/法律等强关系域，图是刚需
EU AI Act	"答案有结构化路径"比纯向量更好审计	监管在推图在金融/医疗加速落地
LazyGraphRAG（微软）	索引成本=向量、全局查询省 700x	重写成本账，上图门槛大降
GraphRAG vs 高级向量（社区争论）	很多"以为要上图"的场景，Contextual Retrieval 就够	先榨干向量，别为上图而上图

老司机说

给你一个不带立场的判断：2026 年，GraphRAG 在金融监管、医疗诊断、跨产品安全这类“强关系 + 强合规”的场景，已经是工业刚需，大厂公开案例年省成本 40-60%。但同时，社区里也有越来越多声音提醒——别神化图。很多你“以为需要图”的场景，先上 Anthropic Contextual Retrieval 把向量榨到极限、再上 LazyGraphRAG 试水，可能就够了。真正的老司机，既知道图什么时候是刚需，也知道它什么时候是杀鸡用牛刀。

WEEK 13 · WRAP UP

Week 13 五节课的总收口——GraphRAG 完整工程体系

让 RAG 从“答得对”升级到“能归纳、能推理”，但只在主场题型发力



关键判断

这五节连起来，就是一套完整的 GraphRAG 工程方法论：先判断该不该上（Boundary），再 schema-first 建图（Build），用三种模式检索（Search），翻译成文本喂给 LLM（Augment），最后用按题型分层的 A/B 证明价值、接入治理（Compare）。一句话——GraphRAG 不再是“实验技术”，而是“特定场景的工业刚需”，但用对的前提是：只在它的主场上场。

WEEK 13 · 收口

Week 13 的 5 个判断——带走这几条，你就能落地 GraphRAG

下周 Week 14: 把数据/索引/Prompt/Skill/图 统一原子绑定

- Boundary: 5 个准入判断决定上不上——2026 先用 Contextual Retrieval, 再上 LazyGraphRAG 试水
- Build: schema-first 是第一性原理——强类型 + LLM 约束抽取 + 4 阶段实体对齐, 漂移域上时序图
- Search: 3 种模式按题型路由 + 向量兜底——2026 用 DRIFT 把 Local/Global 融合
- Augment: 图→文本的翻译质量决定生成质量——3 种序列化 + 3 套模板 + 双链证据
- Compare: A/B 必须按题型分层——37.5% 主场上图、62.5% 客场走向量, 全程接入治理

下一讲 · Week 14 治理与版本: 前 13 周我们把 RAG + Agent + 评测 + 可观测 + GraphRAG 全建出来了, 还差最后一步——把数据/索引/Prompt/Skill/图全部纳入统一版本治理, 实现"原子绑定 + 秒级回滚"。

WEEK 13 · THE END

你的 RAG 从"答得对"升级到"能归纳能推理"

从边界识别到图构建，从 3 种检索模式到图增强生成，再到按题型分层的 A/B——GraphRAG 不再是"实验技术"，而是"特定场景的工业刚需"。用对它的关键，始终是那句话：只在主场上场。

本周交付物（已 push 至 GitHub 仓库）

Graph Schema

pipelines/graph/schema.yaml

Entity Extraction

pipelines/graph/extract.py

Entity Alignment

pipelines/graph/align.py

Community Detection

pipelines/graph/community.py

Graph Search

services/graph/*.py

Query Classifier

services/graph/classifier.py

下周 → Week 14 治理与版本：把数据 / 索引 / Prompt / Skill / 图 统一原子绑定，实现秒级回滚。我们下周见。